Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

Кафедра штучного інтелекту

Дисципліна «Аналітика великих даних»

Практичне заняття №2

«Розподілена обробка великих даних за допомогою Apache Spark»

Виконав:
ст. гр. ДМм-26-1
Зелінський В.С.

Прийняв:
Золотухін О.В.

Харків – 2026

Мета роботи

Мета роботи: ознайомитися з архітектурою та принципами роботи Apache Spark як платформи розподіленої обробки великих даних; отримати практичні навички завантаження, очищення, трансформації та агрегації великого набору даних за допомогою Spark DataFrame і PySpark на прикладі хвилинних свічок BTC/USD (Bitcoin Historical Data, 7 816 537 записів за 2012–2026 роки); навчитися виконувати аналітичні операції над цими даними та використовувати Spark SQL для формування аналітичних запитів, а також порівняти Spark з Hadoop MapReduce за моделлю обчислень, використанням пам'яті, продуктивністю, зручністю програмування та можливостями повторного використання даних.

Короткі теоретичні відомості

Apache Spark – рушій розподілених обчислень, у якому програма-драйвер описує обробку, а виконавці (executors) обробляють частини даних паралельно. У режимі `local[*]` драйвер і виконавець працюють в одному процесі JVM, а паралелізм забезпечують потоки за кількістю ядер, у цій роботі – чотири.

Базова абстракція Spark Core – RDD, незмінна колекція записів, поділена на розділи (partitions); кожен розділ обробляє одна задача (task). Файл обсягом 396,8 МБ Spark ділить на 4 розділи.

DataFrame – надбудова над RDD зі схемою, завдяки якій оптимізатор Catalyst будує фізичний план: виштовхує фільтри, відкидає непотрібні стовпці, обирає стратегію агрегації. DataFrame API і Spark SQL дають той самий план.

Операції поділяються на трансформації (`filter`, `withColumn`, `groupBy`) і дії (`count`, `collect`, `toPandas`). Трансформації ліниві (lazy evaluation): вони лише доповнюють план, а обчислення запускає дія. План перетворюється на DAG, розрізаний на етапи (stages) по межах широких залежностей: вузькі трансформації виконуються конвеєром, а групування чи сортування вимагають обміну (shuffle). Метод `persist()` зберігає

обчислені розділи в пам'яті, що доцільно, коли один DataFrame використовується багатьма діями.

Hadoop MapReduce виконує кожне завдання як пару фаз map і reduce із записом проміжних результатів на диск.

Результати виконання практичного заняття

1 Підготовка середовища

Машина: Intel Xeon 2,10 ГГц (4 логічні ядра), 15 ГБ пам'яті, Ubuntu 24.04; програмне забезпечення і параметри сесії наведено в таблиці 1.

Таблиця 1 – Програмне середовище та параметри SparkSession

Компонент або параметр	Значення
Java (OpenJDK) / Spark / PySpark	21.0.12.1 / 4.3.0 / 4.3.0
Python / pandas / numpy	3.13.18 / 3.0.5 / 2.5.3
Jupyter Notebook / JupyterLab / ipykernel	7.6.5 / 4.6.6 / 7.4.0
spark.master / spark.defaultParallelism	local[*] / 4
spark.driver.memory / spark.sql.shuffle.partitions	8g / 16
spark.sql.session.timeZone / seed	UTC / 42

Розділів обміну 16 (4 на ядро) замість типових 200, бо 200 дрібних задач на 7,8 млн рядків дали б більше накладних витрат, ніж роботи. Зона UTC узгоджує добові межі з ПЗ1. Перевірку показано на рисунку 1; також без помилок виконано `java -version`, `pip show pyspark`, `jupyter --version` і `smoke_test.py` (групування 1000 рядків).

```
bash — pz2

$ python3 -c "import pyspark; ..."  # версії та конфігурація SparkSession
Java                          openjdk version "21.0.12.1" 2026-08-18
Spark                         4.3.0
PySpark                       4.3.0
Python                        3.13.18
pandas                        3.0.5
numpy                         2.5.3
OS                            Linux-6.18.44-fc-v378-x86_64-with-glibc2.39
CPU                           Intel(R) Xeon(R) Processor @ 2.10GHz
CPU_logical_cores             4
RAM_GB                        15
spark.master                  local[*]
spark.defaultParallelism      4
spark.driver.memory           8g
spark.sql.shuffle.partitions  16
spark.sql.session.timeZone    UTC
spark.ui.url                  http://192.0.2.2:4040
seed                          42
```

Рисунок 1 – Версії компонентів і параметри SparkSession

Значення `spark.defaultParallelism 4` підтверджує використання всіх ядер; попередження про неповну підтримку pandas 3.0 на результат не вплинуло.

2 Завантаження даних

Файл прочитано двічі: з `inferSchema=True` і з явною схемою `StructType` (`Timestamp` – `LongType`, решта – `DoubleType`). За `count()` і `printSchema()` DataFrame містить 7 816 537 записів і 6 атрибутів (`Timestamp`, `Open`, `High`, `Low`, `Close`, `Volume`) у 4 розділах; перші рядки 1 січня 2012 року мають однакові ціни 4,58 і нульовий обсяг – заповнені (flat) хвилини. Типи та пропуски зведено в таблицю 2.

Таблиця 2 – Поля DataFrame, типи даних і кількість пропущених значень

Поле	Тип (inferSchema)	Тип (явна схема)	null	NaN
Timestamp	int	bigint	0	0
Open, High, Low, Close, Volume (кожне)	double	double	0	0

Пропусків немає ні як `null`, ні як `NaN`, що узгоджується з ПЗ1. Виведення типів дало для `Timestamp` 32-бітний `int`, бо всі мітки поки менші за 2^31; явний `bigint` не залежить від діапазону і збігається з `int64` у pandas. До того ж завантаження з `count()` з виведенням типів тривало 10,236 с, а з явною схемою – 1,045 с (одиничне вимірювання), бо виведення потребує зайвого повного проходу по файлу.

3 Попередня обробка

3.1 Пропуски, дублікати, типи та похідні атрибути

`dropna()` і `dropDuplicates(["Timestamp"])` (дві свічки з однією міткою суперечили б одна одній) залишили 7 816 537 рядків, тобто 0 дублікатів. Типи приведено явною схемою і `timestamp_seconds`, з якого в UTC виділено `datetime`, `date`, `year`, `month`, `hour`. Також сформовано `is_flat` (нульовий обсяг), `range = High − Low`, `rel_range = (High − Low) / Open`, логарифмічну дохідність `log_ret` і її 60-хвилинне стандартне відхилення `vol60`.

```
w = Window.orderBy("Timestamp")
d = d.withColumn("log_ret", F.log("Close") - F.log(F.lag("Close").over(w)))
```

```
d = d.withColumn("vol60", F.stddev("log_ret").over(w.rowsBetween(-59, 0)))
```

Логарифмічна дохідність адитивна, тому корінь із суми її квадратів за добу дає реалізовану волатильність `rv`. Сітка хвилин повна (П31), тож рамка `rowsBetween(-59, 0)` охоплює рівно 60 хвилин – достатньо для стабільної оцінки σ без згладжування внутрішньодобових сплесків; для діб узято вікна 7 і 30. За `describe()` середня `log_ret` дорівнює 0,0000012 при σ = 0,001733 (≈0,17 % за хвилину), а екстремуми −0,617058 і +0,616914 майже рівні за модулем – ознака одиничного помилкового тику з поверненням. Їх не видалено, бо правила OHLC не порушено, а вплив оцінено в підрозділі 4.2.

3.2 Хвилини з нульовим обсягом

Хвилин з `Volume = 0` – 1 312 419 (16,79 %); у них усі чотири ціни дорівнюють попередньому закриттю, тобто це заповнення сітки, а не угоди. Добову частку показано на рисунку 2.

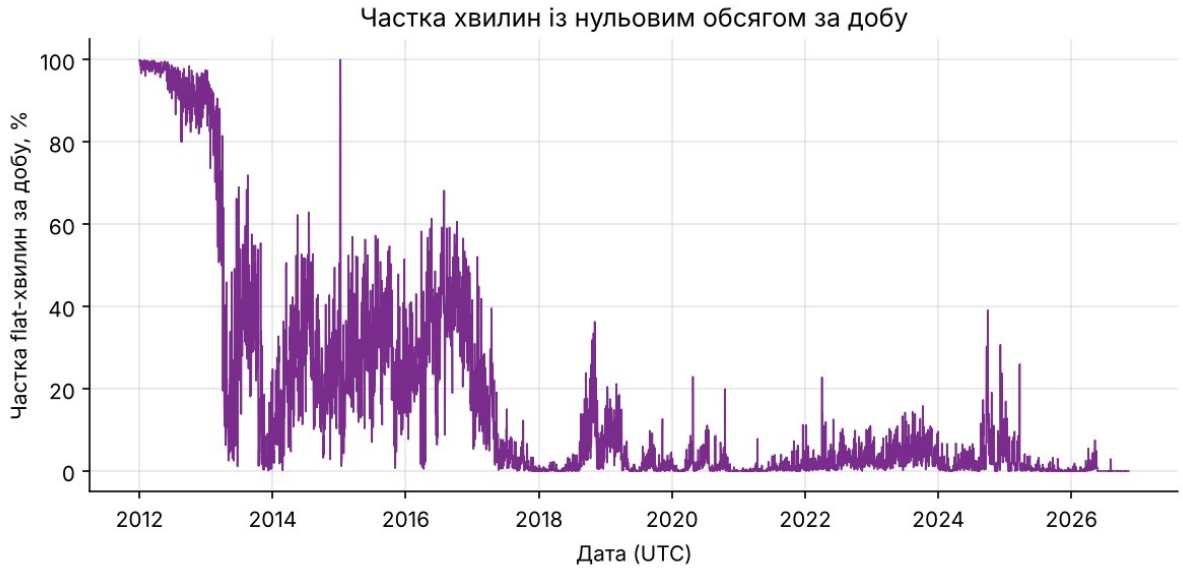


Рисунок 2 – Частка хвилин із нульовим обсягом за добу

Частка становить 94,95 % у 2012 році, 39,16 % у 2013, 33,19 % у 2016 і не перевищує 3,95 % з 2018 року; доби 6–8 січня 2015 року не мають жодної угоди, імовірно через зупинку біржі-джерела. Видалення цих хвилин зруйнувало б зміст `lag("Close")` і 60-хвилинну рамку, тому їх позначено `is_flat` і відфільтровано лише там, де вони спотворюють результат: моменти додатково рахуються для `is_flat = 0`, рейтинги діб –

для `flat_share < 0,5`. Поріг відсікає 2012 рік, зберігає 2013–2016 роки (24,24–39,16 %) і залишає 4876 з 5429 діб.

3.3 Лініві обчислення, план виконання і кешування

Функція `build()` описує весь конвеєр (читання, `dropna`, `dropDuplicates`, десять `withColumn`, два вікна) за 0,245 с, а перша дія `count()` триває 7,829 с: до дії Spark лише будує план. Фізичний план запиту `select("date", "log_ret", "vol60").where("year = 2021")` наведено на рисунку 3.

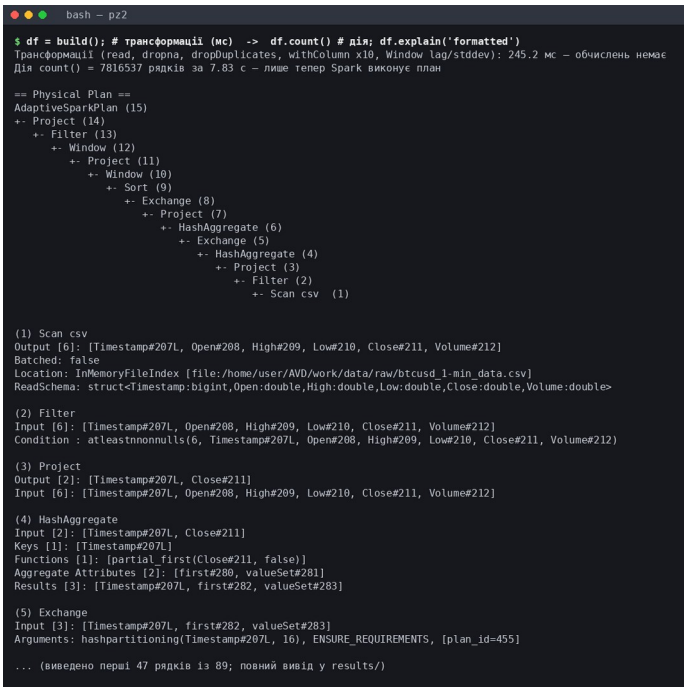


Рисунок 3 – Лініві трансформації, дія count() і фізичний план запиту

Catalyst одразу після `Scan csv` відкидає непотрібні стовпці (`Project (3)`), `dropDuplicates` виконує як `HashAggregate` з обміном `hashpartitioning(Timestamp, 16)`, а фільтр `year = 2021` залишає над вікнами, бо `lag` першої хвилини 2021 року потребує останньої хвилини 2020 року. Недолік – вузол `Exchange SinglePartition`: вікно `orderBy` без `partitionBy` збирає всі 7,8 млн рядків в один розділ і рахує `lag` і `stddev` однією задачею.

Для перевірки кешування запит `heavy_query` (топ-10 діб за `rv` і обсяг за годиною, дві дії) виконано тричі без кешу і тричі після

`persist(StorageLevel.MEMORY_AND_DISK)`, результати наведено в таблиці 3.

Таблиця 3 – Час виконання запиту без кешу і з кешем, с

Режим	Запуск 1	Запуск 2	Запуск 3	Медіана
Без cache()	19,774	17,565	18,349	18,349
З cache()	3,184	2,376	2,329	2,376
Матеріалізація кешу (count)	82,293	–	–	–

Кеш прискорює запит у 7,7 разу, бо прибирає читання CSV, обмін і перерахунок вікон, але матеріалізація (82,293 с) окупається лише після приблизно п'яти повторів; над кешем виконано понад двадцять агрегацій і запитів, тому він виправданий. Вкладка Storage у Spark UI показує один кешований розділ (Cached Partitions = 1) – наслідок `Exchange SinglePartition`, тож і запити до кешу йдуть однією задачею. Загальний час конвеєра 330,477 с прийнятний, тому схему залишено.

4 Розподілена аналітика

4.1 Групування і агрегація

Хвилинні свічки згруповано за датою за правилами OHLC (first/max/min/last/sum); перше й останнє значення взято `min_by`/`max_by` за міткою часу, бо після обміну порядок рядків у групі не гарантований.

```
d1 = df.groupBy("date").agg(
    F.min_by("Open", "Timestamp").alias("open"), F.max("High").alias("high"),
    F.min("Low").alias("low"), F.max_by("Close", "Timestamp").alias("close"),
    F.sum("Volume").alias("volume"),
    F.sqrt(F.sum(F.col("log_ret") ** 2)).alias("rv"))   # також n_minutes,
n_flat
```

Отримано 5429 діб, 5427 з них повні (1440 хвилин), як і в П31. Додано `ret_1d`, `hl_rel = (High − Low) / Open`, частку flat-хвилин, ковзні показники за 7 і 30 діб; так само побудовано погодинний агрегат (130 276 рядків), обидва збережено в Parquet. Групування за годиною доби дало мінімум частки обсягу о 03:00 UTC (2,85 %) і максимум о 14:00 UTC (5,79 %) із синхронною зміною середнього `rel_range` від 0,00067 до 0,00109.

4.2 Статистичні характеристики дохідностей

Асиметрію та надлишковий ексцес (`F.skewness`, `F.kurtosis`) на трьох масштабах наведено в таблиці 4.

Таблиця 4 – Асиметрія та надлишковий ексцес логарифмічних дохідностей

Ряд	n	Ст. відхилення	Асиметрія	Ексцес	Мінімум	Максимум
1 хв, усі свічки	7 816 536	0,001733	3,92	7161,59	−0,6171	0,6169
1 хв, лише з торгами	6 504 118	0,001900	3,58	5958,64	−0,6171	0,6169
1 год, усі	130 275	0,009522	−1,23	118,11	−0,3473	0,2988
1 год, flat_share < 0,5	113 720	0,009235	−1,26	124,73	−0,3473	0,2988
1 доба, усі	5 428	0,040632	−1,31	25,46	−0,6639	0,3375
1 доба, flat_share < 0,5	4 876	0,040541	−1,36	26,70	−0,6639	0,3375

Хвилинний ексцес 7161,59 (для нормального розподілу 0) лише частково зумовлений flat-хвилинами: без них він дорівнює 5958,64. Основну частку четвертого моменту дають хвилини ±0,617 (понад 350 σ) та інші поодинокі тики, тож для моделювання хвилинного ризику викиди слід обмежувати за квантилями. З агрегацією ексцес спадає до 118,11 (година) і 25,46 (доба), а асиметрія стає від'ємною через різкі падіння, зокрема 11.04.2013 (−0,664) і 12.03.2020 (−0,494). Фільтр `flat_share < 0,5` майже нічого не змінює, бо flat-хвилини зосереджені в 2012 році. Форму розподілів показано на рисунку 4.

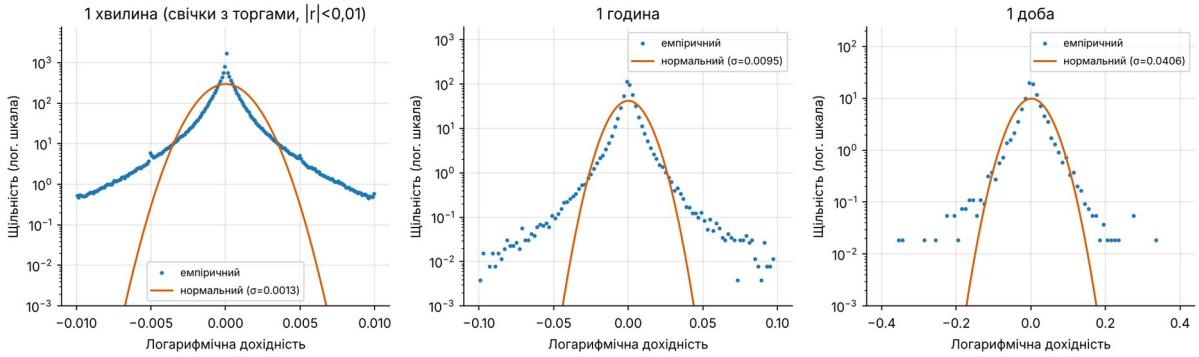


Рисунок 4 – Емпіричні розподіли дохідностей і нормальний розподіл з тим самим σ

На логарифмічній осі нормальний розподіл є параболою, а емпіричний має майже лінійні важкі хвости; при |r| = 0,01 хвилинна

щільність перевищує нормальну на кілька порядків. Добова rv у 2013 році неодноразово перевищує 0,3, після 2023 року лежить у межах 0,01–0,05, а сплески збігаються з подіями (12.03.2020 – 0,211 при 30-денній волатильності 0,094, 19.05.2021 – 0,191 при 0,053).

4.3 Фільтрація і сортування: найволатильніші доби

Повні доби (is_complete = 1) відсортовано за спаданням rv і hl_rel, обидва рейтинги повторено з фільтром flat_share < 0,5. Без фільтра доби 27.01.2012 і 31.01.2012 посідають четверте і шосте місця за rv (0,6358 і 0,6160), хоча 98,1 % і 97,2 % їх хвилин без угод: кожна поодинока угода переносить увесь накопичений рух ціни, тож висока rv тут відображає неліквідність. Після фільтра їх замінюють 18.12.2013 і 20.11.2013 (flat 0,005 і 0,085). Шість із десяти діб за rv припадають на квітень 2013 року (лідер 11.04.2013, rv 1,1260), коли ціна впала з 259,34 до 45 USD. Рейтинги близькі, але не тотожні, бо hl_rel залежить від двох крайніх цін, а rv – від усієї траєкторії; єдина доба після 2013 року за розмахом – 13.03.2020 (44,21 %).

5 Spark SQL

5.1 Тимчасові представлення та запити

Створено представлення btc (кешований хвилинний DataFrame) і btc_daily (добовий агрегат); приклад запиту Q1:

```
df.createOrReplaceTempView("btc"); d1.createOrReplaceTempView("btc_daily")
r = spark.sql("""SELECT year, COUNT(*) AS minutes, ROUND(AVG(Close),2) AS
avg_close,
                 MAX(High) AS max_high, MIN(Low) AS min_low,
ROUND(SUM(Volume),0) AS volume,
                 ROUND(100*AVG(is_flat),2) AS flat_pct FROM btc GROUP BY year
ORDER BY year""")
```

Виконано шість запитів: Q1 – річна статистика; Q2 – десять діб з найбільшим розмахом при flat_share < 0,5; Q3 – середній хвилинний обсяг за днем тижня і годиною з 2018 року; Q4 – місячна сезонність з 2018 року; Q5 – доба з максимальним обсягом у кожному році (ROW_NUMBER() OVER (PARTITION BY year ORDER BY volume DESC)); Q6 – дні обвалів відносно попередніх 30 діб. Завдяки кешу кожен тривав 1,678–2,816 с.

5.2 Результати запитів

Результат Q1 наведено в таблиці 5.

Таблиця 5 – Річна статистика хвилинних свічок (запит Q1)

Рік	Хвилин	Середня ціна, USD	Максимум High	Мінімум Low	Обсяг, BTC	Flat, %
2012	527 039	8,23	16,41	3,8	567 949	94,95
2013	525 600	187,43	1 163,0	12,77	5 031 143	39,16
2014	525 600	525,57	995,0	275,0	5 024 184	24,24
2015	525 600	272,25	502,0	152,4	5 525 344	29,08
2016	527 040	565,83	980,74	352,0	1 992 258	33,19
2017	525 600	3 948,33	19 666,0	751,34	4 698 753	8,04
2018	525 600	7 524,76	17 234,99	3 122,28	3 932 327	3,77
2019	525 600	7 352,19	13 880,0	3 322,19	2 994 600	3,28
2020	527 040	11 074,63	29 300,0	3 850,0	3 074 321	1,55
2021	525 600	47 386,18	69 000,0	27 734,0	1 802 998	0,7
2022	525 600	28 228,66	48 234,0	15 479,0	833 815	2,66
2023	525 600	28 811,76	44 729,0	16 496,0	681 687	3,95
2024	527 040	65 895,12	108 364,0	38 505,0	807 910	3,33
2025	525 600	101 639,98	126 272,0	74 434,0	639 946	1,13
2026	452 378	74 103,97	97 939,0	57 734,63	636 371	0,34

Кількість хвилин відповідає календарю (525 600 і 527 040), неповні лише 2012 рік (ряд з 00:01) і 2026 рік (дані до 11 листопада). Максимум High 126 272 USD припадає на 2025 рік, мінімум 3,8 USD – на 2012 рік. Спад річного обсягу в BTC з 5 525 344 (2015) до 639 946 (2025) не означає падіння інтересу: середня ціна зросла з 272,25 до 101 639,98 USD, а датасет відображає одну біржу.

Q2 повторив рейтинг за розмахом з підрозділу 4.3, що підтверджує еквівалентність DataFrame API і SQL. За Q4 найбільша середня добова `rv` у січні (0,03977) і березні (0,03912), найменша – у жовтні (0,02342). За Q5 рекордна доба за обсягом – 18.12.2013 (137 070 BTC), у 2020 році – 12.03.2020 (58 513 BTC). Результат Q3 наведено на рисунку 5.

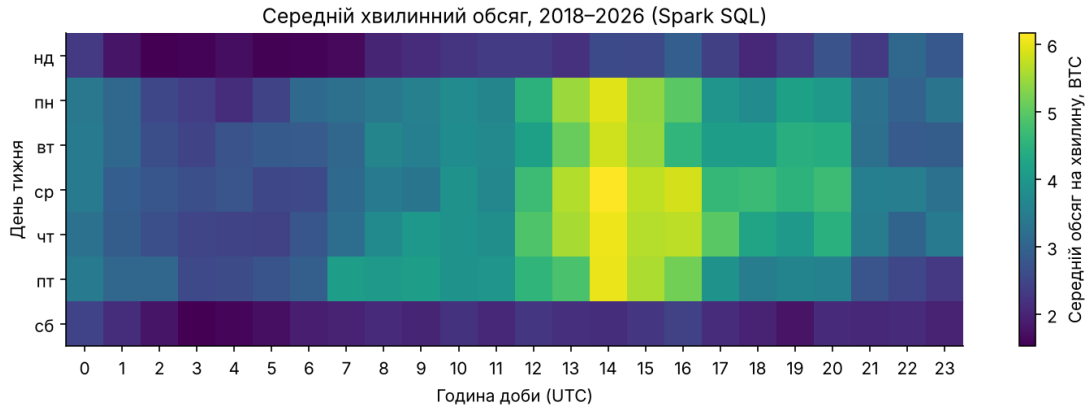


Рисунок 5 – Середній хвилинний обсяг за днем тижня та годиною доби, 2018–2026

Максимум обсягу – у будні о 14:00 UTC (6,1661 BTC на хвилину в середу), вихідні рівномірно низькі (мінімум 1,532 BTC у неділю о 02:00 UTC): з 2018 року активність задають учасники з графіком традиційних ринків. Результат Q6 наведено в таблиці 6.

Таблиця 6 – Доби різких змін ціни: дохідність, волатильність і аномальність обсягу (запит Q6)

Дата	Лог. дохідність, %	rv	Обсяг, BTC	Обсяг / середній за 30 діб
2020-03-12	−49,4	0,2111	58 513	9,27
2020-03-13	14,96	0,2784	54 420	6,81
2021-05-19	−15,34	0,1914	32 469	6,12
2022-06-13	−16,9	0,0938	12 622	6,56
2022-11-08	−10,43	0,0933	12 036	8,45
2022-11-09	−15,56	0,0815	8 351	4,63

У дні обвалу 12–13.03.2020, падіння 19.05.2021 і колапсу FTX 08–09.11.2022 обсяг перевищував середній `AVG(volume)` за 30 попередніх діб у 4,63–9,27 разу, тому сплеск обсягу надійніше вказує на кризу, ніж розмір падіння (08.11.2022: −10,43 % при обсязі у 8,45 разу вищому). Вікно рахується над усіма добами до фільтра за датами.

5.3 План виконання SQL-запиту

Вкладку SQL у Spark UI знято в допоміжній сесії (`spark_ui_sql_shots.py`) без кешу. Граф запиту з групуванням за місяцем показано на рисунку 6.

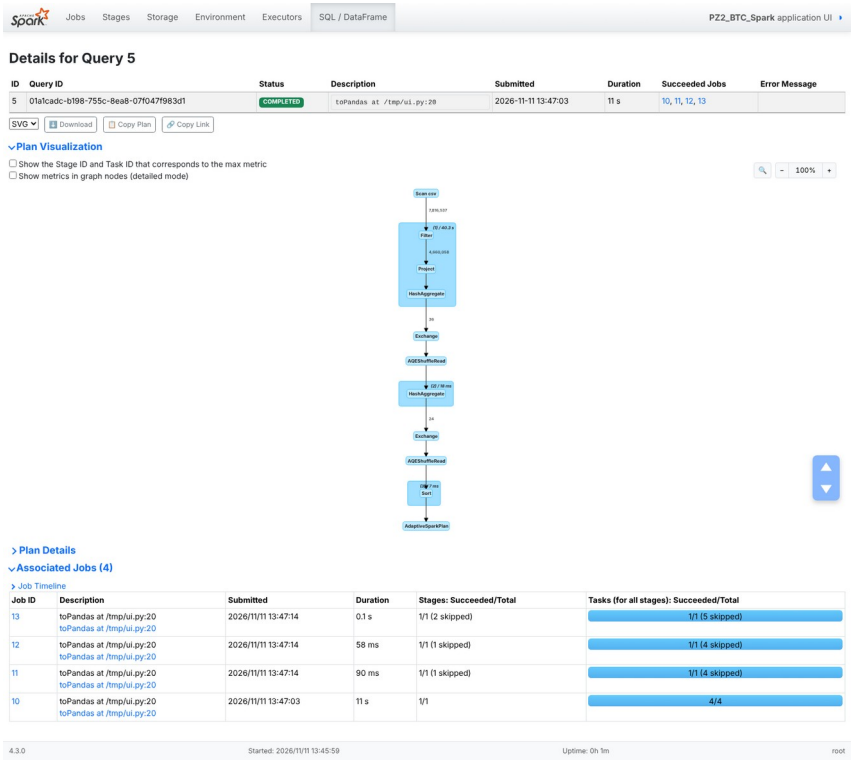


Рисунок 6 – Граф фізичного плану SQL-запиту у Spark UI

Перший етап конвеєром виконує `Scan csv`, `Filter` (`year >= 2018`), `Project` і часткову `HashAggregate` у кожному з 4 розділів, тож `Exchange` передає лише суми по 12 місяцях, а `AQEShuffleRead` об'єднує 16 розділів обміну. Майже весь час іде на розбір CSV, що пояснює виграш від кешу в таблиці 3.

6 Порівняння Hadoop MapReduce та Spark

6.1 Умови та результати вимірювань

Обрано дві однаково формульовані задачі: кількість хвилин і обсяг за роками (мало ключів) і добовий OHLC (5429 ключів). Кожна читає той самий CSV тричі (медіана), базова лінія – pandas, окремо виміряно Spark над кешем; пікову пам'ять (RSS, опитування кожні 50 мс) виміряно окремим запуском. Результати Spark і pandas збігаються. Hadoop MapReduce виконується в ПЗ1 через Hadoop Streaming на тих самих задачах, його результати буде внесено після запуску. Вимірювання наведено в таблиці 7.

Таблиця 7 – Час виконання (медіана з трьох запусків, мінімум–максимум) і пікова пам'ять (один вимірюваний запуск на рушій і задачу)

Задача	Рушій	Медіана, с	Мін.–макс., с	Пікова пам'ять, МБ
Рік: кількість і обсяг	pandas	4,733	4,553–4,760	744,4
Рік: кількість і обсяг	Spark local[*]	3,897	3,159–4,638	1279,2 (JVM) + 127,4 (Python)
Рік: кількість і обсяг	Spark, кеш	1,460	1,410–1,791	–
Рік: кількість і обсяг	Hadoop Streaming	[ЗАПОВНИТИ ПІСЛЯ ЗАПУСКУ]	[ЗАПОВНИТИ ПІСЛЯ ЗАПУСКУ]	[ЗАПОВНИТИ ПІСЛЯ ЗАПУСКУ]
Добовий OHLC	pandas	5,734	5,458–6,243	881,6
Добовий OHLC	Spark local[*]	7,493	5,593–12,414	1478,1 (JVM) + 131,8 (Python)
Добовий OHLC	Spark, кеш	2,190	2,121–2,266	–
Добовий OHLC	Hadoop Streaming	[ЗАПОВНИТИ ПІСЛЯ ЗАПУСКУ]	[ЗАПОВНИТИ ПІСЛЯ ЗАПУСКУ]	[ЗАПОВНИТИ ПІСЛЯ ЗАПУСКУ]

Spark з читанням CSV швидший за pandas лише за роками (3,897 проти 4,733 с) і повільніший на добовому OHLC (7,493 проти 5,734 с): файл 396,8 МБ вміщується в пам'ять, а виграш 4 ядер з'їдають планування етапів, обмін (більший через `min_by`/`max_by` на 5429 ключах) і передача з JVM у Python. Над кешем задачі тривають 1,460 і 2,190 с: порівняння нерівноправне, але показує, що після кешування вартість визначає обчислення, а не читання. Порівняння показано на рисунку 7.

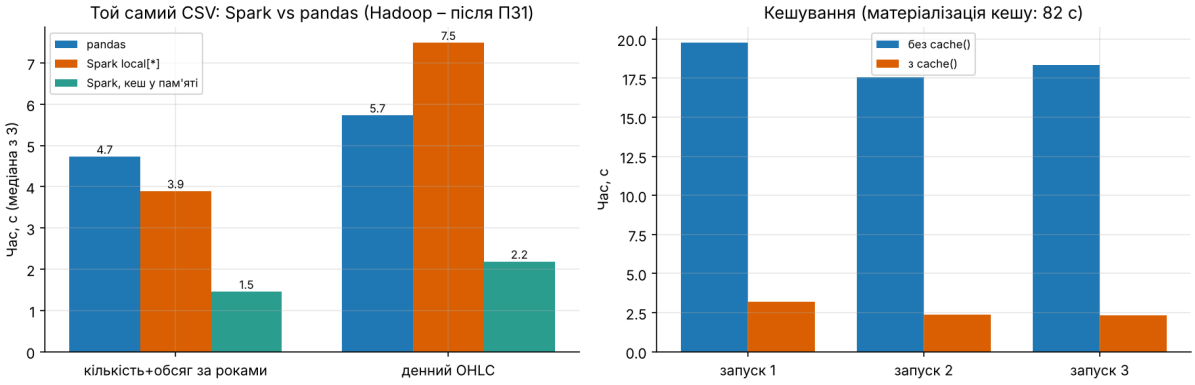


Рисунок 7 – Час виконання задач у Spark і pandas та ефект кешування

Кеш скорочує час задач Spark у 2,7–3,4 разу. За пам'яттю Spark програє (1279,2–1478,1 МБ JVM проти 744,4–881,6 МБ у pandas), але це переважно фіксовані витрати JVM, тоді як потреба pandas зростає з розміром файлу, і файл, більший за пам'ять, Spark обробив би по розділах, а pandas – ні.

6.2 Порівняння за критеріями методичних вказівок

Порівняння за п'ятьма критеріями зведено в таблицю 8: для Spark – виміри цієї роботи, для MapReduce – властивості моделі та виміри П31, які буде внесено після запуску.

Таблиця 8 – Порівняння Hadoop MapReduce і Apache Spark

Критерій	Hadoop MapReduce	Apache Spark (результати П32)
Модель обчислень	Жорстка послідовність map – shuffle – reduce на кожне завдання; багатокроковий аналіз – ланцюжок завдань	DAG з довільної кількості трансформацій; вузькі операції об'єднуються в один етап (рисунок 6: Scan, Filter, Project, HashAggregate в одному етапі)
Використання оперативної пам'яті	Потокова обробка записів, проміжні дані на диску; пікова пам'ять: [ЗАПОВНИТИ ПІСЛЯ ЗАПУСКУ]	Проміжні дані й кеш у пам'яті; JVM драйвера 1279,2–1478,1 МБ на задачах таблиці 7
Продуктивність	Час задач таблиці 7: [ЗАПОВНИТИ ПІСЛЯ ЗАПУСКУ]	3,897 і 7,493 с з читанням CSV, 1,460 і 2,190 с над кешем
Зручність програмування	Окремі програми mapper і reducer, ручний розбір рядків і вибір першого та останнього значення для OHLC	Один вираз `groupBy().agg()` з `min_by`/`max_by` або SQL-запит; DataFrame API і SQL дають однаковий план
Повторне використання даних	Кожне завдання заново читає вхідні дані з HDFS; кешу між завданнями немає	`persist()`: прискорення важкого запиту в 7,7 разу (18,349 → 2,376 с; таблиця 3)

На одній машині ключові переваги Spark – модель DAG без запису проміжних результатів на диск і кешування; кількісний висновок щодо продуктивності: [ЗАПОВНИТИ ПІСЛЯ ЗАПУСКУ]. Виміри на 4 ядрах не відображають масштабування на кластері.

7 Візуалізація

За результатами обробки побудовано дев'ять графіків, чотири з них наведено вище (рисунки 2, 4, 5, 7); решта (ціна закриття, волатильність, рейтинги діб, обсяг за годиною і роками) збережена в `figures`, а їх числові висновки наведено в тексті. У Python передаються лише агрегати (`d1.orderBy("date").toPandas()` – 5429 рядків замість 7 816 537).

Висновки

- Розгорнуто Apache Spark 4.3.0 (`local[*]`, 4 ядра) з PySpark, Java 21, Python 3.13 і Jupyter Notebook 7.6.5; параметри сесії обрано під обсяг даних.
- Датасет завантажено у Spark DataFrame: 7 816 537 записів, 6 атрибутів, 0 пропусків. Явна схема вдесятеро швидша за `inferSchema` (1,045 проти 10,236 с) і задає `Timestamp` як `bigint`.
- Попередня обробка: 0 дублікатів, типи приведено явною схемою, сформовано календарні ознаки, діапазон свічки, дохідність і 60-хвилинну волатильність; хвилини з нульовим обсягом (16,79 %) позначено `is_flat` і враховано фільтрами.
- Реалізовано фільтрацію, групування, сортування, агрегацію та статистичні характеристики: хвилинний ексцес 7161,59 зумовлений переважно викидами ±0,617, для діб він падає до 25,46.
- Над представленнями `btc` і `btc_daily` виконано шість SQL-запитів, які підтвердили обвали 12.03.2020, 19.05.2021 і 08–09.11.2022 з обсягом у 4,63–9,27 разу вищим за середній за 30 діб.
- Модель: Spark виконує вузькі трансформації конвеєром у межах етапу DAG, MapReduce пише результат кожного завдання на диск. Пам'ять: Spark 1279,2–1478,1 МБ JVM проти 744,4–881,6 МБ у pandas, MapReduce – [ЗАПОВНИТИ ПІСЛЯ ЗАПУСКУ]. Продуктивність: Spark 3,897 і 7,493 с проти 4,733 і 5,734 с у pandas, MapReduce – [ЗАПОВНИТИ ПІСЛЯ ЗАПУСКУ]. Зручність: OHLC у Spark – один вираз без mapper і reducer. Повторне використання: кеш прискорив запит у 7,7 разу при матеріалізації 82,293 с.
- Виявлено обмеження конвеєра: вікна без `partitionBy` переносять увесь набір в один розділ; для більших даних їх слід рахувати в розділах за роком з перекриттям, а кеш перерозподіляти.

Набуто навичок розподіленої обробки даних в Apache Spark, роботи з DataFrame і Spark SQL, трансформацій і агрегації.

Додаток А. Лістинг програми

А.1 Файл pz2_spark.py – основний конвеєр обробки (Spark DataFrame, Spark SQL, вимірювання)

```
# -*- coding: utf-8 -*-
"""ПЗ2. Apache Spark (PySpark + Spark SQL) на хвилинних свічках BTC/USD.
Запуск: python3 pz2_spark.py   (результати -> ../results, скріншоти ->
../screenshots)
"""
import os, sys, io, time, platform, subprocess, contextlib, statistics, math
import numpy as np
import pandas as pd

SEED = 42
np.random.seed(SEED)
BASE = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
RES, SHOTS, TXT = f"{BASE}/results", f"{BASE}/screenshots",
f"{BASE}/screenshots/txt"
RAW = "/home/user/AVD/work/data/raw/btcusd_1-min_data.csv"
for d in (RES, SHOTS, TXT):
    os.makedirs(d, exist_ok=True)

from pyspark.sql import SparkSession, Window, functions as F
from pyspark.sql.types import (StructType, StructField, LongType, DoubleType)
from pyspark import StorageLevel
import pyspark

# ---------- допоміжні засоби: тайминги та захоплення виводу для скріншотів
----------
TIMINGS = []
def rec(step, secs, note=""):
    TIMINGS.append((step, round(secs, 3), note))
    print(f"[timing] {step}: {secs:.2f} с {note}")

@contextlib.contextmanager
def timed(step, note=""):
    t = time.perf_counter(); yield
    rec(step, time.perf_counter() - t, note)

class Tee(io.StringIO):
    def write(self, s):
        sys.__stdout__.write(s); return super().write(s)

@contextlib.contextmanager
def cap(name, cmd):
    """Виконує блок, дублює stdout у файл screenshots/txt/<name>.txt з
заголовком '$ команда'."""
    buf = Tee()
    with contextlib.redirect_stdout(buf):
        yield
    with open(f"{TXT}/{name}.txt", "w", encoding="utf-8") as f:
        f.write(f"$ {cmd}\n" + buf.getvalue())

def save(df, name):
    df.to_csv(f"{RES}/{name}.csv", index=False, encoding="utf-8")

def explain_str(df, mode="formatted"):
    return df._sc._jvm.PythonSQLUtils.explainString(df._jdf.queryExecution(),
mode)

# ---------- Завдання 1. Середовище та SparkSession ----------
t0_all = time.perf_counter()
CORES = os.cpu_count()
spark = (SparkSession.builder.appName("PZ2_BTC_Spark")
```

```
        .master("local[*]")                        # усі ядра однієї машини
        .config("spark.driver.memory", "8g")      # 15 ГБ RAM на машині;
решта – ОС і Python
        .config("spark.sql.shuffle.partitions", str(CORES * 4))  # 16: ~4
розділи на ядро, 200 за замовч. надмір для 7,8 млн рядків
        .config("spark.sql.session.timeZone", "UTC")             # усі
часові операції в UTC
        .config("spark.ui.port", "4040")
        .config("spark.ui.showConsoleProgress", "false")
        .getOrCreate())
sc = spark.sparkContext
sc.setLogLevel("ERROR")

def sh(c):
    return subprocess.run(c, shell=True, capture_output=True,
text=True).stdout.strip()
ver_rows = [
    ("Java", sh("java -version 2>&1 | grep -v Picked | head -1")),
    ("Spark", spark.version), ("PySpark", pyspark.__version__),
    ("Python", platform.python_version()), ("pandas", pd.__version__),
    ("numpy", np.__version__), ("OS", platform.platform()),
    ("CPU", sh("lscpu | grep 'Model name' | sed 's/.*: *//'")),
    ("CPU_logical_cores", str(CORES)),
    ("RAM_GB", sh("free -g | awk '/Mem:/{print $2}'")),
    ("spark.master", sc.master), ("spark.defaultParallelism",
str(sc.defaultParallelism)),
    ("spark.driver.memory", spark.conf.get("spark.driver.memory")),
    ("spark.sql.shuffle.partitions",
spark.conf.get("spark.sql.shuffle.partitions")),
    ("spark.sql.session.timeZone",
spark.conf.get("spark.sql.session.timeZone")),
    ("spark.ui.url", sc.uiWebUrl), ("seed", str(SEED)),
]
pd.DataFrame(ver_rows, columns=["parameter",
"value"]).to_csv(f"{RES}/env_versions.csv", index=False)
with cap("01_env", "python3 -c \"import pyspark; ...\"  # версії та
конфігурація SparkSession"):
    for k, v in ver_rows:
        print(f"{k:32s} {v}")

# ---------- Завдання 2. Завантаження даних ----------
SCHEMA = StructType([StructField("Timestamp", LongType()),
StructField("Open", DoubleType()),
                     StructField("High", DoubleType()), StructField("Low",
DoubleType()),
                     StructField("Close", DoubleType()),
StructField("Volume", DoubleType())])

# 2а. Виведення схеми Spark-ом (inferSchema потребує додаткового проходу по
файлу – вимірюємо)
with timed("load_csv_inferSchema_plus_count", "inferSchema=True, count()"):
    raw_inf = spark.read.csv(RAW, header=True, inferSchema=True)
    n_rows = raw_inf.count()
# 2б. Явна схема: без додаткового проходу
with timed("load_csv_explicit_schema_plus_count", "явна схема, count()"):
    raw = spark.read.csv(RAW, header=True, schema=SCHEMA)
    n_rows2 = raw.count()
assert n_rows == n_rows2

with cap("02_schema", "df = spark.read.csv(RAW, header=True,
inferSchema=True); df.printSchema(); df.show(5)"):
    raw_inf.printSchema()
    print("Кількість записів :", n_rows)
    print("Кількість атрибутів:", len(raw_inf.columns))
    print("Назви полів       :", raw_inf.columns)
    print("Типи даних        :", raw_inf.dtypes)
    print("Кількість розділів (partitions) DataFrame:",
```

```
raw_inf.rdd.getNumPartitions())
    raw_inf.show(5)

with timed("missing_values_per_column"):
    nulls = raw.select([F.sum(F.col(c).isNull().cast("int")).alias(c) for c
in raw.columns]).first().asDict()
    nans = raw.select([F.sum(F.isnan(F.col(c)).cast("int")).alias(c) for c in
raw.columns if c != "Timestamp"]).first().asDict()
schema_tbl = pd.DataFrame({"field": raw.columns,
                           "spark_type_inferred": [t for _, t in
raw_inf.dtypes],
                           "spark_type_explicit": [t for _, t in raw.dtypes],
                           "null_count": [nulls[c] for c in raw.columns],
                           "nan_count": [nans.get(c, 0) for c in
raw.columns]})
save(schema_tbl, "pz2_schema_nulls")
pd.DataFrame([("rows", n_rows), ("columns", len(raw.columns)),
              ("raw_partitions_inferSchema", raw_inf.rdd.getNumPartitions()),
              ("raw_partitions_explicit", raw.rdd.getNumPartitions())],
             columns=["metric",
"value"]).to_csv(f"{RES}/pz2_load_summary.csv", index=False)
with cap("03_nulls", "df.select([sum(col(c).isNull()) ...]).show()  #
пропущені значення"):
    print(schema_tbl.to_string(index=False))

# ---------- Завдання 3. Попередня обробка ----------
def build():
    """Повертає (ледачий) DataFrame хвилинних свічок з похідними
атрибутами."""
    d = spark.read.csv(RAW, header=True, schema=SCHEMA)
    d = d.dropna()                                                  # пропущених
немає, але крок виконується
    d = d.dropDuplicates(["Timestamp"])                             # дублікати
за ключем часу
    d = d.withColumn("datetime", F.timestamp_seconds("Timestamp"))   # unix с
-> timestamp (UTC за конфігурацією сесії)
    d = (d.withColumn("date", F.to_date("datetime"))
          .withColumn("year", F.year("datetime")).withColumn("month",
F.month("datetime"))
          .withColumn("hour", F.hour("datetime"))
          .withColumn("is_flat", (F.col("Volume") == 0).cast("int"))      #
нульовий обсяг = заповнена (flat) свічка
          .withColumn("range", F.col("High") - F.col("Low"))               #
діапазон свічки
          .withColumn("rel_range", (F.col("High") - F.col("Low")) /
F.col("Open")))
    w = Window.orderBy("Timestamp")
    d = d.withColumn("log_ret", F.log("Close") -
F.log(F.lag("Close").over(w)))   # логарифмічна дохідність
    # ковзна волатильність: ст. відхилення log_ret за 60 хвилин (сітка без
пропусків, тому 60 рядків = 60 хв)
    d = d.withColumn("vol60", F.stddev("log_ret").over(w.rowsBetween(-59,
0)))
    return d

# 3.1 Демонстрація дублікатів/пропусків
with timed("dedup_check"):
    n_dedup = raw.dropna().dropDuplicates(["Timestamp"]).count()
pd.DataFrame([("rows_raw", n_rows), ("rows_after_dropna",
raw.dropna().count()),
              ("rows_after_dropDuplicates_Timestamp", n_dedup),
              ("duplicates_removed", n_rows - n_dedup)], columns=["metric",
"value"]).to_csv(f"{RES}/pz2_cleaning.csv", index=False)

# ---------- Ледача обчислюваність: трансформації та дії ----------
t = time.perf_counter()
df = build()
```

```
t_transform = time.perf_counter() - t
rec("lazy_build_transformations", t_transform, "лише побудова плану, без
обчислень")
t = time.perf_counter()
n_proc = df.count()                       # дія: запускає виконання всього
плану
t_action = time.perf_counter() - t
rec("first_action_count_uncached", t_action, "count() запускає
читання+вікна")
pd.DataFrame([("transformations_build_s", t_transform),
("first_action_count_s", t_action),
              ("rows", n_proc)], columns=["metric",
"value"]).to_csv(f"{RES}/pz2_lazy_vs_action.csv", index=False)
with cap("04_lazy", "df = build(); # трансформації (мс)  ->  df.count() #
дія; df.explain('formatted')"):
    print(f"Трансформації (read, dropna, dropDuplicates, withColumn x10,
Window lag/stddev): {t_transform*1000:.1f} мс – обчислень немає")
    print(f"Дія count() = {n_proc} рядків за {t_action:.2f} с – лише тепер
Spark виконує план")
    print()
    print(explain_str(df.select('date', 'log_ret', 'vol60').where('year =
2021'), "formatted")[:3800])
with open(f"{RES}/logs/explain_full.txt", "w") as f:
    f.write(explain_str(df, "extended"))

# ---------- Кешування: той самий запит без cache() та з cache() ----------
def heavy_query(d):
    """Два аналітичні запити (дві дії): топ-10 днів за реалізованою
волатильністю та обсяг за годиною доби."""
    a = (d.groupBy("date").agg(F.sqrt(F.sum(F.col("log_ret") **
2)).alias("rv"))
          .orderBy(F.desc("rv")).limit(10).collect())
    b =
d.groupBy("hour").agg(F.sum("Volume").alias("v")).orderBy("hour").collect()
    return a, b

cache_rows = []
df.unpersist()
for i in range(1, 4):
    t = time.perf_counter(); heavy_query(df); dt = time.perf_counter() - t
    cache_rows.append(("without_cache", f"run{i}", round(dt, 3)));
rec(f"cache_exp_without_cache_run{i}", dt)
df.persist(StorageLevel.MEMORY_AND_DISK)
t = time.perf_counter(); df.count(); dt = time.perf_counter() - t       #
матеріалізація кешу
cache_rows.append(("with_cache", "materialize_count", round(dt, 3)));
rec("cache_exp_materialize", dt)
for i in range(1, 4):
    t = time.perf_counter(); heavy_query(df); dt = time.perf_counter() - t
    cache_rows.append(("with_cache", f"run{i}", round(dt, 3)));
rec(f"cache_exp_with_cache_run{i}", dt)
cache_df = pd.DataFrame(cache_rows, columns=["mode", "run", "seconds"])
save(cache_df, "pz2_cache_experiment")
with cap("05_cache", "df.unpersist(); heavy_query(df) x3; df.cache();
df.count(); heavy_query(df) x3"):
    print(cache_df.to_string(index=False))
    m_no = cache_df[(cache_df['mode']=='without_cache')]['seconds'].median()
    m_c = cache_df[(cache_df['mode']=='with_cache') & (cache_df['run']!
='materialize_count')]['seconds'].median()
    print(f"\nмедіана без кешу: {m_no:.2f} с; з кешем: {m_c:.2f} с;
прискорення x{m_no/m_c:.1f}")
    print("Storage level:", df.storageLevel)
pd.DataFrame([("median_without_cache_s", m_no), ("median_with_cache_s", m_c),
("speedup", m_no / m_c)],
             columns=["metric",
"value"]).to_csv(f"{RES}/pz2_cache_summary.csv", index=False)
```

```
# ---------- Опис змінних та контроль якості похідних атрибутів ----------
with timed("describe_derived"):
    desc = df.select("Open", "High", "Low", "Close", "Volume", "range",
"rel_range", "log_ret", "vol60").describe().toPandas()
save(desc, "pz2_describe")
flat = df.agg(F.sum("is_flat").alias("flat"),
F.count("*").alias("n")).first()
flat_by_year = (df.groupBy("year").agg(F.count("*").alias("n_minutes"),
F.sum("is_flat").alias("n_flat"),
                                       F.sum("Volume").alias("volume"),
F.avg("Close").alias("avg_close"))
                  .withColumn("flat_share_pct", F.round(100 * F.col("n_flat")
/ F.col("n_minutes"), 2))
                  .orderBy("year").toPandas())
save(flat_by_year, "pz2_yearly_volume_flat")
pd.DataFrame([("flat_minutes", flat["flat"]), ("rows", flat["n"]),
("flat_share_pct", round(100 * flat["flat"] / flat["n"], 2))],
             columns=["metric",
"value"]).to_csv(f"{RES}/pz2_flat_summary.csv", index=False)
with cap("06_derived",
"df.select('Timestamp','datetime','Close','log_ret','range','rel_range','vol6
0').show(8); df.describe()"):
    df.select("Timestamp", "datetime", "Open", "High", "Low", "Close",
"Volume", "log_ret", "range", "rel_range", "vol60", "year", "month",
"hour").filter("year = 2021 and month = 5").show(8, truncate=False)
    print(desc.to_string(index=False))
    print(f"\nНульовий обсяг: {flat['flat']} з {flat['n']} =
{100*flat['flat']/flat['n']:.2f} %")

# ---------- Завдання 4. Розподілена аналітика ----------
# 4.1 Денний агрегат (групування + агрегація) з похідними ознаками
with timed("daily_aggregation"):
    d1 = (df.groupBy("date").agg(
        F.min_by("Open", "Timestamp").alias("open"),
F.max("High").alias("high"), F.min("Low").alias("low"),
        F.max_by("Close", "Timestamp").alias("close"),
F.sum("Volume").alias("volume"),
        F.count("*").alias("n_minutes"), F.sum("is_flat").alias("n_flat"),
        F.sqrt(F.sum(F.col("log_ret") ** 2)).alias("rv"),            #
реалізована волатильність за добу
        F.stddev("log_ret").alias("std_ret_1m"),
F.avg("vol60").alias("vol60_mean"),
        F.max("rel_range").alias("max_rel_range_1m")))
    wd = Window.orderBy("date")
    d1 = (d1.withColumn("ret_1d", F.log("close") -
F.log(F.lag("close").over(wd)))
            .withColumn("hl_rel", (F.col("high") - F.col("low")) /
F.col("open"))
            .withColumn("flat_share", F.col("n_flat") / F.col("n_minutes"))
            .withColumn("is_complete", (F.col("n_minutes") ==
1440).cast("int"))
            .withColumn("log_volume", F.log1p("volume"))
            .withColumn("vol_7d", F.stddev("ret_1d").over(wd.rowsBetween(-6,
0)))
            .withColumn("vol_30d", F.stddev("ret_1d").over(wd.rowsBetween(-
29, 0)))
            .withColumn("ma_7", F.avg("close").over(wd.rowsBetween(-6, 0)))
            .withColumn("ma_30", F.avg("close").over(wd.rowsBetween(-29, 0)))
            .withColumn("dow", F.dayofweek("date")).withColumn("year",
F.year("date")).withColumn("month", F.month("date")))
    daily = d1.orderBy("date").toPandas()
daily["date"] = pd.to_datetime(daily["date"])
daily.to_parquet(f"{RES}/pz2_daily_features.parquet", index=False)
save(daily, "pz2_daily_features")

# 4.2 Найволатильніші дні (сортування + фільтрація)
comp = d1.filter("is_complete = 1")
```

```
with timed("top10_volatile_days"):
    top_rv = comp.orderBy(F.desc("rv")).limit(10).toPandas()
    top_hl = comp.orderBy(F.desc("hl_rel")).limit(10).toPandas()
    # варіант без «розріджених» днів: щонайменше 50 % хвилин з торгами (щоб
виключити 2012–2013 з частковими заповненнями)
    top_rv_active = comp.filter("flat_share <
0.5").orderBy(F.desc("rv")).limit(10).toPandas()
    top_hl_active = comp.filter("flat_share <
0.5").orderBy(F.desc("hl_rel")).limit(10).toPandas()
cols = ["date", "open", "high", "low", "close", "volume", "rv", "hl_rel",
"ret_1d", "flat_share"]
save(top_rv[cols], "pz2_top10_rv"); save(top_hl[cols], "pz2_top10_hl_rel")
save(top_rv_active[cols], "pz2_top10_rv_active"); save(top_hl_active[cols],
"pz2_top10_hl_rel_active")
with cap("07_top_days", "comp.orderBy(desc('rv')).limit(10);
comp.orderBy(desc('hl_rel')).limit(10)"):
    pd.set_option("display.width", 200)
    print("Топ-10 за реалізованою волатильністю rv = sqrt(sum(log_ret^2)),
повні доби:")
    print(top_rv[cols].round(4).to_string(index=False))
    print("\nТоп-10 за (High-Low)/Open:")
    print(top_hl[cols].round(4).to_string(index=False))
    print("\nТоп-10 за rv, flat_share < 0,5:")
    print(top_rv_active[cols].round(4).to_string(index=False))

# 4.3 Обсяг за годиною доби (UTC) та за роками
with timed("volume_by_hour"):
    vh = (df.groupBy("hour").agg(F.sum("Volume").alias("volume_total"),

F.avg("Volume").alias("volume_mean_per_minute"),
                                 F.avg(F.when(F.col("is_flat") == 0,
F.col("Volume"))).alias("volume_mean_active_minute"),
                                 F.avg("rel_range").alias("rel_range_mean"),
                                 F.stddev("log_ret").alias("std_ret_1m"))
            .withColumn("share_pct", 100 * F.col("volume_total") /
F.sum("volume_total").over(Window.partitionBy())).orderBy("hour").toPandas())
save(vh, "pz2_volume_by_hour")
save(flat_by_year, "pz2_volume_by_year")
with cap("08_volume", "df.groupBy('hour').agg(sum('Volume')...);
df.groupBy('year').agg(...)"):
    print(vh.round(5).to_string(index=False)); print();
print(flat_by_year.round(2).to_string(index=False))

# 4.4 Погодинна таблиця ознак
with timed("hourly_features"):
    hh = (df.groupBy(F.date_trunc("hour",
"datetime").alias("datetime_utc")).agg(
        F.min_by("Open", "Timestamp").alias("open"),
F.max("High").alias("high"), F.min("Low").alias("low"),
        F.max_by("Close", "Timestamp").alias("close"),
F.sum("Volume").alias("volume"),
        F.count("*").alias("n_minutes"), F.sum("is_flat").alias("n_flat"),
        F.sqrt(F.sum(F.col("log_ret") ** 2)).alias("rv"),
F.stddev("log_ret").alias("std_ret_1m")))
    wh = Window.orderBy("datetime_utc")
    hh = (hh.withColumn("ret_1h", F.log("close") -
F.log(F.lag("close").over(wh)))
            .withColumn("range", F.col("high") -
F.col("low")).withColumn("rel_range", (F.col("high") - F.col("low")) /
F.col("open"))
            .withColumn("flat_share", F.col("n_flat") / F.col("n_minutes"))
            .withColumn("log_volume", F.log1p("volume"))
            .withColumn("vol_24h", F.stddev("ret_1h").over(wh.rowsBetween(-
23, 0)))
            .withColumn("hour", F.hour("datetime_utc")).withColumn("dow",
F.dayofweek("datetime_utc"))
            .withColumn("year", F.year("datetime_utc")))
```

```
    hourly = hh.orderBy("datetime_utc").toPandas()
hourly.to_parquet(f"{RES}/pz2_hourly_features.parquet", index=False)

# 4.5 Асиметрія та ексцес дохідностей на різних масштабах
def moments(name, d, col, filt=None):
    x = d.filter(filt) if filt else d
    r = x.filter(F.col(col).isNotNull()).agg(F.count(col).alias("n"),
F.avg(col).alias("mean"), F.stddev(col).alias("std"),
                                             F.skewness(col).alias("skew"),
F.kurtosis(col).alias("excess_kurtosis"),
                                             F.min(col).alias("min"),
F.max(col).alias("max")).first().asDict()
    r["series"] = name; return r
with timed("skew_kurtosis"):
    rows = [moments("1 хв, усі свічки", df, "log_ret"),
            moments("1 хв, лише з торгами (Volume>0)", df, "log_ret",
"is_flat = 0"),
            moments("1 год, усі",
d1.sparkSession.createDataFrame(hourly[["ret_1h"]].dropna()), "ret_1h"),
            moments("1 год, flat_share<0,5",
d1.sparkSession.createDataFrame(hourly[hourly.flat_share < 0.5]
[["ret_1h"]].dropna()), "ret_1h"),
            moments("1 доба, усі",
d1.sparkSession.createDataFrame(daily[["ret_1d"]].dropna()), "ret_1d"),
            moments("1 доба, flat_share<0,5",
d1.sparkSession.createDataFrame(daily[daily.flat_share < 0.5]
[["ret_1d"]].dropna()), "ret_1d")]
mom = pd.DataFrame(rows)[["series", "n", "mean", "std", "skew",
"excess_kurtosis", "min", "max"]]
save(mom, "pz2_skew_kurtosis")
with cap("09_moments", "F.skewness('log_ret'), F.kurtosis('log_ret')  # 1
хв / 1 год / 1 доба"):
    print(mom.to_string(index=False, float_format=lambda v: f"{v:.6g}"))

# гістограма хвилинних дохідностей (лише з торгами) для порівняння з
нормальним розподілом
BW = 0.0001
with timed("minute_return_histogram"):
    hist = (df.filter("is_flat = 0 and log_ret is not null and abs(log_ret) <
0.01")
              .withColumn("bin", F.floor(F.col("log_ret") /
BW)).groupBy("bin").count().orderBy("bin").toPandas())
hist["bin_center"] = (hist["bin"] + 0.5) * BW
save(hist, "pz2_hist_ret_1m_active")

# ---------- Завдання 5. Spark SQL ----------
df.createOrReplaceTempView("btc")
d1.createOrReplaceTempView("btc_daily")
SQLQ = {
 "q1_year_stats": """SELECT year, COUNT(*) AS minutes, ROUND(AVG(Close),2) AS
avg_close, MAX(High) AS max_high, MIN(Low) AS min_low,
                           ROUND(SUM(Volume),0) AS volume,
ROUND(100*AVG(is_flat),2) AS flat_pct
                    FROM btc GROUP BY year ORDER BY year""",
 "q2_top_days_range": """SELECT date, open, high, low,
ROUND((high-low)/open,4) AS hl_rel, ROUND(volume,0) AS volume,
ROUND(flat_share,3) AS flat_share
                    FROM btc_daily WHERE is_complete = 1 AND flat_share <
0.5 ORDER BY hl_rel DESC LIMIT 10""",
 "q3_dow_hour_volume": """SELECT dayofweek(datetime) AS dow, hour,
ROUND(AVG(Volume),4) AS avg_volume_per_minute,
                           ROUND(SUM(Volume),0) AS volume_total
                    FROM btc WHERE year >= 2018 GROUP BY dow, hour ORDER BY
dow, hour""",
 "q4_month_seasonality": """SELECT month, ROUND(AVG(rv),5) AS avg_daily_rv,
ROUND(AVG(hl_rel),4) AS avg_hl_rel, ROUND(AVG(volume),1) AS avg_daily_volume,
COUNT(*) AS days
```

```
                    FROM btc_daily WHERE year >= 2018 AND is_complete = 1
GROUP BY month ORDER BY month""",
 "q5_max_volume_day_per_year": """SELECT year, date, ROUND(volume,0) AS
volume, close FROM (
                        SELECT year, date, volume, close, ROW_NUMBER() OVER
(PARTITION BY year ORDER BY volume DESC) AS rn FROM btc_daily)
                    WHERE rn = 1 ORDER BY year""",
 "q6_crash_days_vs_30d_mean": """SELECT date, ROUND(100*ret_1d,2) AS ret_pct,
ROUND(rv,4) AS rv, ROUND(volume,0) AS volume,
                         ROUND(volume / AVG(volume) OVER (ORDER BY date ROWS
BETWEEN 30 PRECEDING AND 1 PRECEDING),2) AS volume_x_prev30_mean
                    FROM btc_daily WHERE date IN (DATE'2020-03-12',
DATE'2020-03-13', DATE'2021-05-19', DATE'2022-11-08', DATE'2022-11-09',
DATE'2022-06-13') ORDER BY date""",
}
# q6: вікно рахується всередині відфільтрованого набору, тому рахуємо у
підзапиті над усіма днями
SQLQ["q6_crash_days_vs_30d_mean"] = """SELECT * FROM (
    SELECT date, ROUND(100*ret_1d,2) AS ret_pct, ROUND(rv,4) AS rv,
ROUND(volume,0) AS volume,
           ROUND(volume / AVG(volume) OVER (ORDER BY date ROWS BETWEEN 30
PRECEDING AND 1 PRECEDING),2) AS volume_x_prev30_mean
    FROM btc_daily) WHERE date IN (DATE'2020-03-12', DATE'2020-03-13',
DATE'2021-05-19', DATE'2022-06-13', DATE'2022-11-08', DATE'2022-11-09') ORDER
BY date"""
for i, (k, q) in enumerate(SQLQ.items(), 1):
    with timed(f"sql_{k}"):
        r = spark.sql(q).toPandas()
    save(r, f"pz2_sql_{k}")
    with cap(f"10_sql_{k}", "spark.sql(\"" + " ".join(q.split())[:150] +
" ...\").show()"):
        print(r.head(25).to_string(index=False))
with open(f"{RES}/pz2_sql_queries.sql", "w", encoding="utf-8") as f:
    for k, q in SQLQ.items():
        f.write(f"-- {k}\n{' '.join(q.split())};\n\n")
with cap("11_sql_explain", "spark.sql('SELECT year, ... FROM btc GROUP BY
year').explain()"):
    print(explain_str(spark.sql(SQLQ["q1_year_stats"]), "formatted")[:3500])

# ---------- Завдання 6. Spark vs pandas vs (Hadoop) на тому самому CSV
----------
def spark_year_count_volume():
    d = spark.read.csv(RAW, header=True, schema=SCHEMA)
    return
d.groupBy(F.year(F.timestamp_seconds("Timestamp")).alias("year")).agg(F.count
("*").alias("n"), F.sum("Volume").alias("vol")).orderBy("year").toPandas()
def spark_daily_ohlc():
    d = spark.read.csv(RAW, header=True, schema=SCHEMA).withColumn("date",
F.to_date(F.timestamp_seconds("Timestamp")))
    return (d.groupBy("date").agg(F.min_by("Open",
"Timestamp").alias("open"), F.max("High").alias("high"),
F.min("Low").alias("low"),
                                  F.max_by("Close",
"Timestamp").alias("close"),
F.sum("Volume").alias("volume")).orderBy("date").toPandas())
def pandas_year_count_volume():
    d = pd.read_csv(RAW); y = pd.to_datetime(d["Timestamp"],
unit="s").dt.year
    return d.groupby(y).agg(n=("Volume", "size"), vol=("Volume", "sum"))
def pandas_daily_ohlc():
    d = pd.read_csv(RAW); dt = pd.to_datetime(d["Timestamp"],
unit="s").dt.date
    return d.groupby(dt).agg(open=("Open", "first"), high=("High", "max"),
low=("Low", "min"), close=("Close", "last"), volume=("Volume", "sum"))
cmp_rows = []
chk = {}
for task, fs, fp in (("year_count_volume", spark_year_count_volume,
```

```
pandas_year_count_volume),
                    ("daily_ohlc", spark_daily_ohlc, pandas_daily_ohlc)):
    for engine, fn in (("spark_local[*]", fs), ("pandas", fp)):
        for run in (1, 2, 3):
            t = time.perf_counter(); out = fn(); dt = time.perf_counter() - t
            cmp_rows.append((task, engine, run, round(dt, 3), "читання CSV +
агрегація"))
            chk[(task, engine)] = out
    # перевірка однаковості результатів
    a, b = chk[(task, "spark_local[*]")], chk[(task, "pandas")]
    if task == "year_count_volume":
        assert (a["n"].values == b["n"].values).all() and
np.allclose(a["vol"].values, b["vol"].values)
    else:
        assert len(a) == len(b) and np.allclose(a["close"].values,
b["close"].values) and np.allclose(a["volume"].values, b["volume"].values)
# Spark на кешованому DataFrame (дані вже в пам'яті, без читання CSV)
for run in (1, 2, 3):
    t = time.perf_counter()
    df.groupBy("year").agg(F.count("*"),
F.sum("Volume")).orderBy("year").collect(); dt1 = time.perf_counter() - t
    cmp_rows.append(("year_count_volume", "spark_cached_df", run, round(dt1,
3), "лише агрегація над кешем"))
    t = time.perf_counter()
    df.groupBy("date").agg(F.min_by("Open", "Timestamp"), F.max("High"),
F.min("Low"), F.max_by("Close", "Timestamp"), F.sum("Volume")).collect(); dt2
= time.perf_counter() - t
    cmp_rows.append(("daily_ohlc", "spark_cached_df", run, round(dt2, 3),
"лише агрегація над кешем"))
cmp_rows.append(("year_count_volume", "hadoop_streaming", "", "[ЗАПОВНИТИ
ПІСЛЯ ЗАПУСКУ]", "значення з ПЗ1 (Colab)"))
cmp_rows.append(("daily_ohlc", "hadoop_streaming", "", "[ЗАПОВНИТИ ПІСЛЯ
ЗАПУСКУ]", "значення з ПЗ1 (Colab)"))
cmp = pd.DataFrame(cmp_rows, columns=["task", "engine", "run", "seconds",
"note"])
save(cmp, "pz2_engine_comparison")
num = cmp[pd.to_numeric(cmp["seconds"], errors="coerce").notna()].copy();
num["seconds"] = num["seconds"].astype(float)
summ = num.groupby(["task", "engine"])["seconds"].agg(["median", "min",
"max"]).round(3).reset_index()
save(summ, "pz2_engine_comparison_summary")
for _, r in summ.iterrows():
    rec(f"engine_{r['task']}_{r['engine']}", r["median"], "медіана з 3
запусків")
rec("engine_year_count_volume_hadoop_streaming", float("nan"), "[ЗАПОВНИТИ
ПІСЛЯ ЗАПУСКУ]")
rec("engine_daily_ohlc_hadoop_streaming", float("nan"), "[ЗАПОВНИТИ ПІСЛЯ
ЗАПУСКУ]")
with cap("12_engines", "spark vs pandas: рік (count+volume) та денний OHLC, 3
запуски, той самий CSV"):
    print(cmp.to_string(index=False)); print();
print(summ.to_string(index=False))

# ---------- Знімки Spark UI, поки сесія жива ----------
def ui_screens():
    os.environ.setdefault("PLAYWRIGHT_BROWSERS_PATH", "/opt/pw-browsers")
    from playwright.sync_api import sync_playwright
    pages = {"jobs": "jobs/", "stages": "stages/", "sql": "SQL/",
"executors": "executors/", "storage": "storage/", "environment":
"environment/"}
    with sync_playwright() as p:
        b =
p.chromium.launch(executable_path="/opt/pw-browsers/chromium_headless_shell-
1243/chrome-headless-shell-linux64/chrome-headless-shell")
        pg = b.new_page(viewport={"width": 1500, "height": 950})
        for k, u in pages.items():
            pg.goto(f"http://localhost:4040/{u}", wait_until="load");
```

```
pg.wait_for_timeout(1200)
            pg.screenshot(path=f"{SHOTS}/spark_ui_{k}.png")
        # деталі найважчого SQL-запиту (план виконання)
        pg.goto("http://localhost:4040/SQL/", wait_until="load")
        link = pg.locator("table a").first
        if link.count():
            link.click(); pg.wait_for_timeout(1500);
pg.screenshot(path=f"{SHOTS}/spark_ui_sql_detail.png", full_page=False)
        b.close()
try:
    ui_screens()
except Exception as e:
    print("UI screenshots failed:", repr(e))
    open(f"{RES}/logs/ui_error.txt", "w").write(repr(e))

rec("TOTAL_script_runtime", time.perf_counter() - t0_all)
pd.DataFrame(TIMINGS, columns=["step", "seconds",
"note"]).to_csv(f"{RES}/timings.csv", index=False)
spark.stop()
print("OK")
```

A.2 Файл make_figures.py – побудова рисунків

```
# -*- coding: utf-8 -*-
"""Рисунки до ПЗ2 з файлів results/ (matplotlib, dpi 150, підписи
українською)."""
import os, numpy as np, pandas as pd, matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
np.random.seed(42)
BASE = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
R, FG = f"{BASE}/results", f"{BASE}/figures"
plt.rcParams.update({"font.size": 10, "axes.grid": True, "grid.alpha": .3,
"axes.spines.top": False, "axes.spines.right": False})
C = ["#1f77b4", "#d95f02", "#2a9d8f", "#7b2d8e"]
def out(n): plt.tight_layout(); plt.savefig(f"{FG}/{n}.png", dpi=150);
plt.close()
daily = pd.read_parquet(f"{R}/pz2_daily_features.parquet"); hourly =
pd.read_parquet(f"{R}/pz2_hourly_features.parquet")

# 1. Ціна закриття (лог-шкала) + обсяг
fig, ax = plt.subplots(2, 1, figsize=(10, 6), sharex=True,
gridspec_kw={"height_ratios": [3, 1]})
ax[0].semilogy(daily.date, daily.close, color=C[0], lw=.9, label="Close
(кінець доби)")
ax[0].semilogy(daily.date, daily.ma_30, color=C[1], lw=.9, label="ковзне
середнє 30 діб")
ax[0].set_ylabel("Ціна, USD (лог. шкала)"); ax[0].legend();
ax[0].set_title("Ціна закриття BTC/USD, добові дані зі Spark")
ax[1].bar(daily.date, daily.volume, width=1, color=C[2]);
ax[1].set_ylabel("Обсяг за добу, BTC"); ax[1].set_xlabel("Дата (UTC)")
out("fig01_close_log")

# 2. Розподіл дохідностей vs нормальний (лог. вісь Y)
from math import erf
h = pd.read_csv(f"{R}/pz2_hist_ret_1m_active.csv"); mk =
pd.read_csv(f"{R}/pz2_skew_kurtosis.csv")
fig, ax = plt.subplots(1, 3, figsize=(13, 4))
def panel(a, x, bw, title, xlab, lim):
    x = x[np.isfinite(x)]; mu, sd = x.mean(), x.std()
    bins = np.arange(-lim, lim + bw, bw); cnt, e = np.histogram(x,
bins=bins); ce = (e[1:] + e[:-1]) / 2
    dens = cnt / (len(x) * bw); ok = cnt > 0
    a.semilogy(ce[ok], dens[ok], ".", color=C[0], ms=4, label="емпіричний")
    g = np.linspace(-lim, lim, 400); a.semilogy(g, np.exp(-(g - mu) ** 2 / (2
* sd ** 2)) / (sd * np.sqrt(2 * np.pi)), color=C[1], label=f"нормальний
```

```
(σ={sd:.4f})")
    a.set_title(title); a.set_xlabel(xlab); a.set_ylabel("Щільність (лог.
шкала)"); a.legend(fontsize=8); a.set_ylim(bottom=1e-3)
x1 = np.repeat(h.bin_center.values, h["count"].values)
panel(ax[0], x1, 1e-4, "1 хвилина (свічки з торгами, |r|<0,01)",
"Логарифмічна дохідність", 0.01)
panel(ax[1], hourly.ret_1h.dropna().values, 0.002, "1 година", "Логарифмічна
дохідність", 0.1)
panel(ax[2], daily.ret_1d.dropna().values, 0.01, "1 доба", "Логарифмічна
дохідність", 0.4)
out("fig02_returns_vs_normal")

# 3. Обсяг за годиною доби
vh = pd.read_csv(f"{R}/pz2_volume_by_hour.csv")
fig, ax = plt.subplots(figsize=(9, 4)); ax.bar(vh.hour, vh.share_pct,
color=C[0])
ax.set_xlabel("Година доби (UTC)"); ax.set_ylabel("Частка сумарного обсягу,
%"); ax.set_xticks(range(24))
ax.set_title("Розподіл торгового обсягу за годиною доби, 2012–2026");
out("fig03_volume_by_hour")

# 4. Волатильність у часі
fig, ax = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
ax[0].plot(daily.date, daily.rv, color=C[0], lw=.5, label="добова реалізована
волатильність rv")
ax[0].plot(daily.date, daily.vol_30d, color=C[1], lw=1.2, label="σ добових
дохідностей, вікно 30 діб")
ax[0].set_yscale("log"); ax[0].set_ylabel("Волатильність (лог. шкала)");
ax[0].legend(fontsize=8)
ax[1].plot(daily.date, daily.vol60_mean, color=C[2], lw=.7);
ax[1].set_yscale("log")
ax[1].set_ylabel("Середня за добу σ(60 хв)"); ax[1].set_xlabel("Дата (UTC)")
ax[0].set_title("Волатильність BTC/USD: добова rv та ковзна σ за 60 хвилин");
out("fig04_volatility")

# 5. Обсяг за роками + частка flat
y = pd.read_csv(f"{R}/pz2_yearly_volume_flat.csv")
fig, ax = plt.subplots(figsize=(9, 4.2)); ax.bar(y.year, y.volume / 1e6,
color=C[0]); ax.set_xlabel("Рік"); ax.set_ylabel("Обсяг, млн BTC")
a2 = ax.twinx(); a2.plot(y.year, y.flat_share_pct, color=C[1], marker="o");
a2.set_ylabel("Частка хвилин з нульовим обсягом, %"); a2.grid(False)
ax.set_xticks(y.year); ax.tick_params(axis="x", rotation=45);
ax.set_title("Річний обсяг і частка заповнених (flat) хвилин");
out("fig05_yearly_volume")

# 6. Топ-10 днів
t = pd.read_csv(f"{R}/pz2_top10_rv.csv"); t2 =
pd.read_csv(f"{R}/pz2_top10_hl_rel_active.csv")
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
ax[0].barh(t.date[::-1], t.rv[::-1], color=C[0]);
ax[0].set_xlabel("Реалізована волатильність rv"); ax[0].set_title("Топ-10 за
rv (повні доби)")
ax[1].barh(t2.date[::-1], t2.hl_rel[::-1] * 100, color=C[1]);
ax[1].set_xlabel("(High–Low)/Open, %"); ax[1].set_title("Топ-10 за розмахом
(flat < 50 %)")
out("fig06_top_days")

# 7. Погодинна й добова сезонність (теплова карта день тижня × година)
q = pd.read_csv(f"{R}/pz2_sql_q3_dow_hour_volume.csv").pivot(index="dow",
columns="hour", values="avg_volume_per_minute")
fig, ax = plt.subplots(figsize=(10, 3.6)); im = ax.imshow(q.values,
aspect="auto", cmap="viridis")
ax.set_yticks(range(7)); ax.set_yticklabels(["нд", "пн", "вт", "ср", "чт",
"пт", "сб"]); ax.set_xticks(range(24)); ax.grid(False)
ax.set_xlabel("Година доби (UTC)"); ax.set_ylabel("День тижня");
plt.colorbar(im, label="Середній обсяг на хвилину, BTC")
ax.set_title("Середній хвилинний обсяг, 2018–2026 (Spark SQL)");
```

```
out("fig07_dow_hour_heatmap")

# 8. Час виконання: Spark vs pandas, кеш
s = pd.read_csv(f"{R}/pz2_engine_comparison_summary.csv"); c =
pd.read_csv(f"{R}/pz2_cache_experiment.csv")
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
tasks = ["year_count_volume", "daily_ohlc"]; names = {"year_count_volume":
"кількість+обсяг за роками", "daily_ohlc": "денний OHLC"}
w = .26
for i, (e, lab) in enumerate([("pandas", "pandas"), ("spark_local[*]", "Spark
local[*]"), ("spark_cached_df", "Spark, кеш у пам'яті")]):
    v = [s[(s.task == k) & (s.engine == e)]["median"].iloc[0] for k in tasks]
    b = ax[0].bar(np.arange(2) + (i - 1) * w, v, w, label=lab, color=C[i])
    for r_, vv in zip(b, v): ax[0].text(r_.get_x() + r_.get_width() / 2, vv,
f"{vv:.1f}", ha="center", va="bottom", fontsize=8)
ax[0].set_xticks(range(2)); ax[0].set_xticklabels([names[k] for k in tasks]);
ax[0].set_ylabel("Час, с (медіана з 3)"); ax[0].legend(fontsize=8)
ax[0].set_title("Той самий CSV: Spark vs pandas (Hadoop – після ПЗ1)")
cc = c[c["run"] != "materialize_count"].copy(); cc["lab"] =
cc["mode"].map({"without_cache": "без cache()", "with_cache": "з cache()"})
for i, m in enumerate(["without_cache", "with_cache"]):
    v = cc[cc["mode"] == m].seconds.values; ax[1].bar(np.arange(3) + (i - .5)
* .38, v, .38, color=C[i], label=cc[cc["mode"] == m].lab.iloc[0])
mat = c[c["run"] == "materialize_count"].seconds.iloc[0]
ax[1].set_xticks(range(3)); ax[1].set_xticklabels(["запуск 1", "запуск 2",
"запуск 3"]); ax[1].set_ylabel("Час, с"); ax[1].legend(fontsize=8)
ax[1].set_title(f"Кешування (матеріалізація кешу: {mat:.0f} с)");
out("fig08_timings")

# 9. Об'ємна модель: залежність волатильності від рівня flat (чому flat слід
враховувати)
fig, ax = plt.subplots(figsize=(8, 4)); ax.plot(daily.date, daily.flat_share
* 100, color=C[3], lw=.8)
ax.set_ylabel("Частка flat-хвилин за добу, %"); ax.set_xlabel("Дата (UTC)");
ax.set_title("Частка хвилин із нульовим обсягом за добу");
out("fig09_flat_share")
print("ok")
```

A.3 Файл make_screens.py – рендер виводу програми в зображення терміналу

```
# -*- coding: utf-8 -*-
"""Рендер реального виводу програми (screenshots/txt/*.txt) у PNG: темний
фон, моноширинний шрифт."""
import glob, os
from PIL import Image, ImageDraw, ImageFont
BASE = os.path.abspath(os.path.join(os.path.dirname(__file__), ".."))
FONT = "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf"
BOLD = "/usr/share/fonts/truetype/dejavu/DejaVuSansMono-Bold.ttf"
SZ, PAD, MAXLINES, MAXCOLS = 16, 18, 48, 150
f, fb = ImageFont.truetype(FONT, SZ), ImageFont.truetype(BOLD, SZ)
cw = f.getlength("M"); lh = SZ + 5
for p in sorted(glob.glob(f"{BASE}/screenshots/txt/*.txt")):
    lines = open(p, encoding="utf-8").read().rstrip("\n").split("\n")
    lines = [l.expandtabs(4) for l in lines]
    note = None
    if len(lines) > MAXLINES:
        note = f"... (виведено перші {MAXLINES - 1} рядків із {len(lines)};
повний вивід у results/)"
        lines = lines[:MAXLINES - 1]
    lines = [(l[:MAXCOLS - 1] + "…") if len(l) > MAXCOLS else l for l in
lines]
    if note: lines.append(note)
    w = int(max(len(l) for l in lines) * cw + 2 * PAD) + 10
    h = lh * (len(lines)) + 2 * PAD + 34
    im = Image.new("RGB", (max(w, 700), h), (22, 24, 29)); d =
```

```
ImageDraw.Draw(im)
    d.rectangle([0, 0, im.width, 30], fill=(48, 52, 62))
    for i, c in enumerate([(255, 95, 86), (255, 189, 46), (39, 201, 63)]):
        d.ellipse([12 + i * 22, 9, 24 + i * 22, 21], fill=c)
    d.text((100, 6), "bash – pz2", font=f, fill=(190, 195, 205))
    y = 30 + PAD
    for i, l in enumerate(lines):
        if i == 0 and l.startswith("$"):
            d.text((PAD, y), "$", font=fb, fill=(120, 220, 120)); d.text((PAD
+ cw * 2, y), l[2:], font=fb, fill=(240, 240, 240))
        else:
            d.text((PAD, y), l, font=f, fill=(205, 210, 220))
        y += lh
    out = f"{BASE}/screenshots/term_{os.path.basename(p)[:-4]}.png"
    im.save(out); print(out)
```

A.4 Файл measure_memory.py – вимірювання пікової пам'яті pandas і Spark

```
# -*- coding: utf-8 -*-
"""Пік пам'яті (RSS) для pandas і Spark на тих самих двох задачах, що й у
pz2_spark.py.
Кожне вимірювання – окремий процес; psutil опитує RSS кожні 50 мс (процес +
діти, для Spark – JVM драйвера)."""
import os, sys, time, subprocess, threading, psutil, csv
RAW = "/home/user/AVD/work/data/raw/btcusd_1-min_data.csv"
RES = os.path.abspath(os.path.join(os.path.dirname(__file__), "..",
"results"))
WORKER = r'''
import sys, pandas as pd
RAW, eng, task = sys.argv[1:4]
if eng == "pandas":
    d = pd.read_csv(RAW)
    if task == "year_count_volume":
        y = pd.to_datetime(d["Timestamp"], unit="s").dt.year
        r = d.groupby(y).agg(n=("Volume", "size"), vol=("Volume", "sum"))
    else:
        dt = pd.to_datetime(d["Timestamp"], unit="s").dt.date
        r = d.groupby(dt).agg(open=("Open","first"), high=("High","max"),
low=("Low","min"), close=("Close","last"), volume=("Volume","sum"))
else:
    from pyspark.sql import SparkSession, functions as F
    from pyspark.sql.types import *
    spark =
(SparkSession.builder.master("local[*]").config("spark.driver.memory","8g")
             .config("spark.sql.shuffle.partitions","16").config("spark.sql.s
ession.timeZone","UTC")
             .config("spark.ui.enabled","false").getOrCreate())
    SCHEMA = StructType([StructField("Timestamp", LongType())] +
[StructField(c, DoubleType()) for c in
["Open","High","Low","Close","Volume"]])
    d = spark.read.csv(RAW, header=True, schema=SCHEMA)
    if task == "year_count_volume":
        r =
d.groupBy(F.year(F.timestamp_seconds("Timestamp")).alias("year")).agg(F.count
("*").alias("n"), F.sum("Volume").alias("vol")).orderBy("year").toPandas()
    else:
        d = d.withColumn("date", F.to_date(F.timestamp_seconds("Timestamp")))
        r = d.groupBy("date").agg(F.min_by("Open","Timestamp").alias("open"),
F.max("High").alias("high"), F.min("Low").alias("low"),
             F.max_by("Close","Timestamp").alias("close"),
F.sum("Volume").alias("volume")).orderBy("date").toPandas()
    spark.stop()
print("rows", len(r))
'''
def run(eng, task):
```

```
    p = subprocess.Popen([sys.executable, "-I", "-c", WORKER, RAW, eng,
task], stdout=subprocess.PIPE, text=True)
    ps = psutil.Process(p.pid); peak = {"py": 0, "jvm": 0, "tot": 0}
    while p.poll() is None:
        try:
            py = ps.memory_info().rss; jvm = 0
            for c in ps.children(recursive=True):
                try:
                    if "java" in c.name(): jvm = max(jvm,
c.memory_info().rss)
                except psutil.Error: pass
            peak["py"] = max(peak["py"], py); peak["jvm"] = max(peak["jvm"],
jvm); peak["tot"] = max(peak["tot"], py + jvm)
        except psutil.Error: pass
        time.sleep(0.05)
    return peak
rows = []
for eng in ["pandas", "spark"]:
    for task in ["year_count_volume", "daily_ohlc"]:
        pk = run(eng, task); mb = lambda b: round(b / 2**20, 1)
        if eng == "pandas":
            rows.append((eng, task, mb(pk["py"]), "peak RSS of Python process
(psutil, 50 ms polling)"))
        else:
            rows.append((eng, task, mb(pk["jvm"]), "peak RSS of driver JVM
(java child process, psutil, 50 ms polling), local[*], driver 8g"))
            rows.append(("spark_python_client", task, mb(pk["py"]), "peak RSS
of PySpark Python process (excluding JVM)"))
        print(rows[-1], flush=True)
with open(f"{RES}/pz2_memory.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f); w.writerow(["engine", "task", "peak_mb", "method"]);
w.writerows(rows)
```

A.5 Файл smoke_test.py – перевірка працездатності PySpark

```
from pyspark.sql import SparkSession
spark =
SparkSession.builder.master("local[2]").config("spark.ui.enabled","false").ge
tOrCreate()
spark.sparkContext.setLogLevel("ERROR")
df = spark.range(1, 1001).selectExpr("id", "id % 7 AS k")
print("Spark", spark.version, "| count =", df.count())
df.groupBy("k").count().orderBy("k").show()
spark.stop()
```

A.6 Файл spark_ui_sql_shots.py – знімки вкладки SQL у Spark UI

```
import sys, time, re
sys.path.insert(0, "/tmp")
from pyspark.sql import SparkSession
from playwright.sync_api import sync_playwright
src = open("/home/user/AVD/work/pz2/code/pz2_spark.py", encoding="utf-
8").read()
RAW = "/home/user/AVD/work/data/raw/btcusd_1-min_data.csv"
spark =
(SparkSession.builder.appName("PZ2_BTC_Spark").master("local[*]").config("spa
rk.driver.memory","8g")
         .config("spark.sql.shuffle.partitions","16").config("spark.sql.sessi
on.timeZone","UTC").config("spark.ui.port","4040").getOrCreate())
spark.sparkContext.setLogLevel("ERROR")
from pyspark.sql import functions as F
d = spark.read.csv(RAW, header=True, inferSchema=True)
d = (d.withColumn("datetime",
F.timestamp_seconds("Timestamp")).withColumn("year",
F.year("datetime")).withColumn("month", F.month("datetime"))
      .withColumn("hour", F.hour("datetime")).withColumn("is_flat",
(F.col("Open")==F.col("Close")).cast("int")))
```

```
d.createOrReplaceTempView("btc")
qs = re.findall(r"^-- q[123].*\n(.*);",
open("/home/user/AVD/work/pz2/results/pz2_sql_queries.sql").read(), re.M)
for q in [
 "SELECT year, COUNT(*) AS minutes, ROUND(AVG(Close),2) AS avg_close,
MAX(High) AS max_high, MIN(Low) AS min_low, ROUND(SUM(Volume),0) AS volume,
ROUND(100*AVG(is_flat),2) AS flat_pct FROM btc GROUP BY year ORDER BY year",
 "SELECT dayofweek(datetime) AS dow, hour, ROUND(AVG(Volume),4) AS
avg_volume_per_minute, ROUND(SUM(Volume),0) AS volume_total FROM btc WHERE
year >= 2018 GROUP BY dow, hour ORDER BY dow, hour",
 "SELECT month, COUNT(*) AS minutes, ROUND(AVG(Volume),4) AS avg_vol FROM btc
WHERE year >= 2018 GROUP BY month ORDER BY month"]:
    print(spark.sql(q).toPandas().shape, flush=True)
url = spark.sparkContext.uiWebUrl; print(url)
out = "/home/user/AVD/work/pz2/screenshots"
with sync_playwright() as p:
    b = p.chromium.launch(executable_path="/opt/pw-browsers/chromium-1243/
chrome-linux64/chrome", args=["--no-sandbox"])
    pg = b.new_page(viewport={"width":1500,"height":900})
    pg.goto(url + "/SQL/", wait_until="load")
    pg.wait_for_selector("table tbody tr", timeout=30000); time.sleep(3)
    pg.screenshot(path=f"{out}/spark_ui_sql.png", full_page=True)
    links = pg.eval_on_selector_all("a[href*='SQL/execution/?id=']",
"e=>e.map(a=>a.href)")
    print(links)
    pg.goto(links[0], wait_until="load")
    pg.wait_for_selector("svg", timeout=30000); time.sleep(3)
    pg.screenshot(path=f"{out}/spark_ui_sql_detail.png", full_page=True)
    b.close()
spark.stop()
```